

Demo 2 — Khan SRBCT cancer classification

$p = 2308$ genes, 4 cancer types, $n = 83$ — and the answer is twelve genes

Table of contents

The data	1
Load the data	2
R	2
Python	2
Fit COMBSS	2
R	2
Python	3
How many genes does COMBSS pick?	3
R	3
Python	3
Test-set accuracy at each k	3
R	3
Python	4
The headline	4
Reference	4

R code is executed; outputs below are real. Click the **Python** tab for the equivalent script.

The data

Khan et al. (2001) profiled gene expression for small round blue-cell tumours, a family of four childhood cancers that look nearly identical under a microscope:

- **BL** — Burkitt lymphoma
- **EWS** — Ewing's sarcoma
- **NB** — neuroblastoma
- **RMS** — rhabdomyosarcoma

Each tumour is described by **2308 cDNA microarray probes**. The training set has 63 samples; the test set has 20. The classical version of the question is: *can you train a classifier that distinguishes the four tumour types using only a handful of genes?*

The CSVs in `data/` are formatted with the class label in the first column (`y`) and the 2308 probes in columns `X1`–`X2308`.

Load the data

R

```
train <- read.csv("../data/Khan_train.csv")
test  <- read.csv("../data/Khan_test.csv")

x_train <- as.matrix(train[, -1]); y_train <- train$y
x_test  <- as.matrix(test[, -1]); y_test  <- test$y

dim(x_train); table(y_train)
```

```
[1] 63 2308
```

```
y_train
 1  2  3  4
8 23 12 20
```

Python

```
import pandas as pd

train = pd.read_csv("../data/Khan_train.csv")
test  = pd.read_csv("../data/Khan_test.csv")

X_train = train.drop(columns="y").to_numpy()
y_train = train["y"].to_numpy()
X_test  = test.drop(columns="y").to_numpy()
y_test  = test["y"].to_numpy()

print(X_train.shape)
print(pd.Series(y_train).value_counts())
```

Fit COMBSS

Run COMBSS for $k = 1, \dots, 20$ with multinomial logistic loss and the test set as validation:

R

```
library(combss)
fit_khan <- combss(x_train, y_train,
                   x_val = x_test, y_val = y_test,
                   family = "multinomial", q = 20)
fit_khan
```

```
COMBSS fit
family:    multinomial
n, p:      63, 2308
q:         20
lam_ridge: 0
best k:    12
accuracy:   1
subset:     187, 246, 509, 545, 910, 1074, 1319, 1389, 1645, 1954, 1955, 2050
```

Python

```
from combss import combss

fit_khan = combss(X_train, y_train,
                  X_val=X_test, y_val=y_test,
                  family="multinomial", q=20)
print(fit_khan)
```

How many genes does COMBSS pick?

R

```
fit_khan$best_k
```

```
[1] 12
```

```
fit_khan$subset_list[[ fit_khan$best_k ]]
```

```
[1] 187 246 509 545 910 1074 1319 1389 1645 1954 1955 2050
```

Python

```
print(fit_khan.best_k)
print(fit_khan.subset_list[fit_khan.best_k - 1])
```

Test-set accuracy at each k

R

```
plot(seq_along(fit_khan$accuracy_path), fit_khan$accuracy_path, type = "b", pch = 19,
     xlab = "k (number of selected genes)", ylab = "Test accuracy")
abline(v = fit_khan$best_k, lty = 2)
```

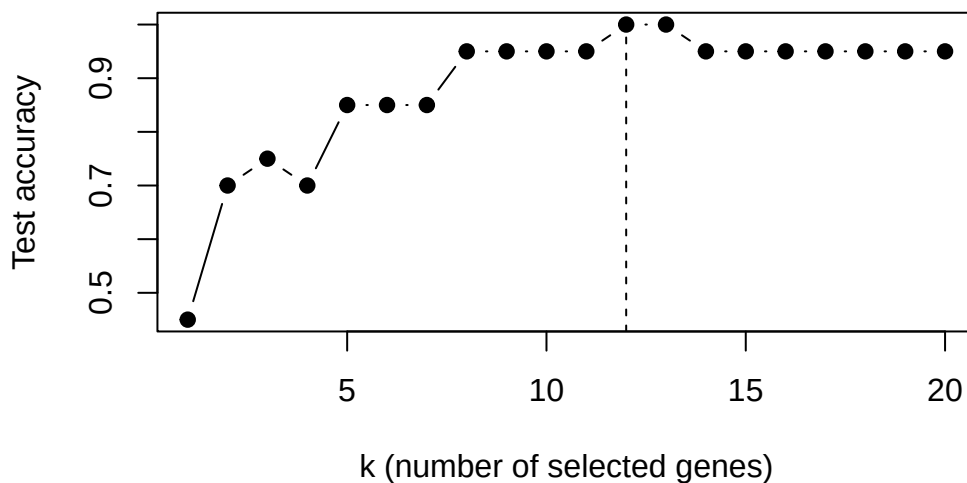


Figure 1: Accuracy as a function of the number of selected genes.

Python

```
import matplotlib.pyplot as plt
plt.plot(range(1, len(fit_khan.accuracy_path) + 1), fit_khan.accuracy_path, "o-")
plt.axvline(fit_khan.best_k, ls="--")
plt.xlabel("k (number of selected genes)")
plt.ylabel("Test accuracy")
plt.show()
```

The headline

Twelve genes are enough to separate the four tumour types — perfect classification on the held-out 20 samples, drawn from 2308 candidates. The R and Python implementations agree on the same twelve genes.

A lasso baseline on the same training data selects an order of magnitude more genes for comparable accuracy (see [Comparisons](#)).

Reference

Khan, J. et al. (2001). *Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks*. **Nature Medicine** 7, 673–679.